

Dualer Studiengang B. Sc. Informatik

Erarbeitungsphase

Technical Software Design

Jana Meyer: 10236813

Lena Pannbacker: 1240698

Dozent: Christoph Schöning

Abgabedatum: 31.09.2026

# Ausgewählte Probleme / Verletzungen

## 1. TaskManager verletzt SRP

- Wo: task\_manager.py
- Problem:
  - TaskManager übernimmt gleichzeitig:
    - Task-Erstellung
    - Validierung
    - Persistenzzugriff
    - Nutzerzugriff
    - Benachrichtigungslogik
    - Reminder-Logik
    - Formatierung / Statuslogik
- Einordnung
  - SOLID: SRP verletzt
  - Kohäsion: zu niedrig
  - Kopplung: zu hoch
  - Code Smell: God Object / Large Class
- Refactoring-vorschlag
  - TaskManager aufteilen in z. B.:
    - TaskService → fachliche Task-Operationen
    - TaskRepository → Speichern/Laden von Tasks
    - NotificationService → Benachrichtigungen
    - ReminderService → Erinnerungslogik

## 2. TaskManager hängt direkt an Database, UserManager, NotificationCenter → DIP verletzt

- Wo:
  - task\_manager.py, Konstruktor
  - `self.db = Database()`  
`self.email = EmailService()`  
`self.notif = NotificationCenter()`  
`self.users = UserManager()`

- Problem:
  - Die Klasse hängt direkt an konkreten Klassen statt an Abstraktionen.  
→ verletzt das Dependency Inversion Principle (DIP).
  - Folgen:
    - schwer testbar
    - schwer austauschbar
    - jede Änderung an Infrastruktur schlägt direkt in die Geschäftslogik durch
  - Beispiel: Wenn JSON durch SQLite ersetzt wird, muss TaskManager mitangepasst werden.
- Einordnung
  - SOLID: DIP verletzt
  - Architektur: Domänen-/Application-Logik kennt Infrastrukturdetails
  - Kopplung: sehr hoch
- Refactoring
  - Abhängigkeiten injizieren:
  - class TaskService:
 

```
def __init__(self, task_repository, user_repository, notification_service):
    self.task_repository = task_repository
    self.user_repository = user_repository
    self.notification_service = notification_service
```
  - Interfaces / Protokolle definieren:
    - TaskRepository
    - UserRepository
    - Notifier

### 3. Database mischt Persistenz und Validierung → Schichtverletzung / SRP verletzt

- Wo?
  - database.py
- Problem
  - Database sollte eigentlich nur Persistenz machen. Tatsächlich macht die Klasse aber zusätzlich:
    - Validierung von Tasks
    - Validierung von Usern
    - Logging
    - Dateistrukturverwaltung
    - Laden und Speichern von zwei unterschiedlichen Domänenobjekten

- Beispiel:
  - `if task.get("title") is None or task.get("title") == "":`  
   `log_error("Task ohne Titel kann nicht gespeichert werden")`  
   `return False`
  - `if task.get("priority", 0) < 1 or task.get("priority", 0) > 3:`  
   `log_error("Ungueltige Prioritaet")`  
   `return False`
  - Das ist Geschäftslogik und gehört nicht in die Persistenzschicht.
- Einordnung
  - SOLID: SRP verletzt
  - Architekturverletzung: Infrastruktur übernimmt Domänenlogik
  - Information Hiding: Schichten sind unsauber getrennt
- Refactoring
  - Trennen in:
    - `TaskRepository / UserRepository` → nur speichern/laden
    - Validierung in `TaskService` oder Domänenobjekt
    - Logging ggf. außen oder über zentralen Logger
- 

## 4. NotificationCenter verletzt Open/Closed Principle

- Wo?
  - `notifications.py`
- Problem
  - Für jeden neuen Kanal (fax, slack, teams, ...) müsste die bestehende `notify()`-Methode geändert werden:
    - `if channel == "email":`  
   `...`  
   `elif channel == "sms":`  
   `...`  
   `elif channel == "push":`  
   `...`
  - Das System ist nicht offen für Erweiterung, sondern nur durch Modifikation erweiterbar.
- Einordnung
  - SOLID: OCP verletzt
  - Code Smell: große if/elif-Kette
  - Erweiterbarkeit: schwach
- Refactoring
  - Polymorphie / Strategy Pattern:
    - `class Notifier:`  
   `def send(self, user, subject, body): ...`

```
class EmailNotifier(Notifier): ...
class SmsNotifier(Notifier): ...
class PushNotifier(Notifier): ...
```

- Dann eine Registry:
  - `notifiers = {`
    - `"email": EmailNotifier(),`
    - `"sms": SmsNotifier(),`
    - `"push": PushNotifier(),`  - `}`

## 5. EmailService verschickt auch SMS und Push → irreführender Name + SRP verletzt

- Wo?
  - `email_service.py`
- Problem
  - Die Klasse heißt `EmailService`, verschickt aber:
    - E-Mails
    - SMS
    - Push-Nachrichten
    - `def send(self, ...)`
      - `def send_sms(self, ...)`
      - `def send_push(self, ...)`
  - das ist sowohl Naming-Problem als auch SRP-Verletzung.
    - ➔ der Name verspricht eine Verantwortlichkeit, die Klasse hat aber drei.
- Einordnung
  - Code Smell: irreführender Name / niedrige Kohäsion
  - SOLID: SRP verletzt
- Refactoring
  - `EmailSender`
  - `SmsSender`
  - `PushSender`

## 6. UserManager ist zu eng an Database gekoppelt und vermischt fachliche Verwaltung mit Objektfabrik → verletzt SRP

- Wo?

- user.py
- Problem
  - UserManager macht gleichzeitig:
    - Persistenzzugriff
    - User-Erstellung
    - Rollenänderung
    - Berechtigungslogik
    - Fabrik für User, AdminUser, ReadOnlyUser
    - Änderungs-/Löschlogik
  - zu viel Verantwortung in einer Klasse
- Einordnung
  - SOLID: SRP verletzt
  - Kohäsion: zu niedrig
  - Architektur: Vermischung von Service, Repository und Factory
- Refactoring
  - Trennen in:
    - UserRepository
    - UserService
    - UserFactory oder Rollenmapper

## 7. UserManager.build\_user\_obj() + user\_types.py brechen Liskov / erzeugen inkonsistentes Verhalten

- Wo?
  - user.py, user\_types.py
- Problem
  - User.update\_email() gibt True zurück  
AdminUser.update\_email() setzt nur self.em, aber gibt nichts zurück  
ReadOnlyUser.update\_email() wirft eine Exception
  - Ähnlich bei delete\_account():
    - User.delete\_account() → True
    - AdminUser.delete\_account() → Exception
    - ReadOnlyUser.delete\_account() → Exception
  - Das ist problematisch, weil Unterklassen sich nicht wie die Basisklasse verhalten.
- Einordnung

- SOLID: LSP verletzt
- API-Design: inkonsistente Verträge
- Testbarkeit: schlecht
- Refactoring
  - Saubere Domänenregeln:
    - Entweder Rückgabevertrag überall gleich
    - oder überall Exceptions
    - oder Fähigkeiten nicht über Vererbung modellieren, sondern über Rollen/Policies
  - Beispiel:
    - `UserPolicy.can_delete(user)`
    - `UserPolicy.can_change_email(user)`

## 8. Zu breite implizite Schnittstellen, es gibt keine Interfaces

- Wo?
  - Gesamter Code
- Problem
  - Unnötige Abhängigkeiten
  - Schlechte Austauschbarkeit
- Einordnung
  - SOLID: verletzt ISP
- Refactoring
  - z. B. Adapter Pattern für Notifier

## 9. Persistenz-Skalierung: Jeder Speichervorgang überschreibt beide JSON-Dateien

- Wo?
  - `Database.py`
- Problem
  - Schlechte Skalierbarkeit
  - Ineffizient
- Einordnung
  - Information Hiding
  - DIP
  - SRP
  - ISP
- Refactoring
  - Speichern hinter Repository-Interface legen
  - Task- und User-Speicher trennen

## 10. Harte Magic Numbers für Prioritäten und Benachrichtigungsmodi

- Wo?
  - task\_manager.py, config.py, utils.py
  - Beispiele:
    - Prioritäten: 1, 2, 3
    - Benachrichtigungsmodus: 1, 2, 3, 4
    - if prio == 1:  
...  
elif prio == 2:  
...  
elif prio == 3:  
...    - if mode == 1:  
...  
elif mode == 2:  
...  
elif mode == 3:  
...  
elif mode == 4:
- Problem
  - Die Zahlen haben ohne Kontext keine Bedeutung.  
Das erschwert Lesbarkeit, Wartung und Erweiterung.
- Einordnung
  - Code Smell: Magic Numbers
  - Verbalisierung: schlechte Ausdruckskraft
  - Abstraktion: fachliche Konzepte werden nicht sauber modelliert
- Refactoring
  - Enums oder Konstanten einführen:
  - from enum import Enum

```
class Priority(Enum):  
    LOW = 1  
    MEDIUM = 2  
    HIGH = 3
```

```
class NotificationMode(Enum):  
    EMAIL = "email"  
    SMS = "sms"  
    BOTH = "both"  
    ALL = "all"
```

# Architektur

## ADR-001: Hexagonales Architekturmodell

Status: vorgeschlagen

Kontext: (Warum mussten wir entscheiden?)

- Die in den Datei-Headern behaupteten Schichten werden nicht eingehalten
- Es gibt keine Abstraktionen
- Es entstehen mehrere Database Instanzen mit inkonsistenten Zuständen

Entscheidung: (Warum haben wir entschieden?)

- Domain: Task, User, Priority, Status; Ports als TaskRepository, UserRepository, Notifier
- Application: TaskService, TaskRepository, UserRepository bekommen ihre Ports per Konstruktor injiziert
- Infrastructure: Adapter: JsonTaskRepository, SmtplibNotifier
- Main.py: Composition Root: Hier werden die Klassen erzeugt und verdrahtet

Schichtendiagramm:

TODO

Alternativen: (Was haben wir verworfen und warum?)

- MVC: TaskMaster hat keine View --> verworfen
- Schichten: ohne Ports bleibt die Abhängigkeit Application – Infrastructure bestehen --> verworfen

Konsequenzen: (Was wird leichter? Was wird schwerer?)

- + Kernlogik testbar
- + Ein Änderungsgrund pro Klasse, Architektur im Code sichtbar
- Mehr Dateien, höhere Komplexität
- Für diese Projektgröße ist der Aufwand grenzwertig

## ODER

## ADR-001: TaskManager in Services aufteilen

Status: vorgeschlagen

Kontext: (Warum mussten wir entscheiden?)

- TaskManager hat sieben Verantwortlichkeiten: Erstellung, Validierung, Persistenz, Nutzerzugriff, Benachrichtigung, Reminder und Formatierung
- Ist nicht isoliert testbar
- God-Objekt

Entscheidung: (Warum haben wir entschieden?)

Aufteilung nach Verantwortlichkeit:

| Einheit | Schicht | Verantwortung             |
|---------|---------|---------------------------|
| Task    | Domain  | Regeln, Selbstvalidierung |

|                     |                |                            |
|---------------------|----------------|----------------------------|
| TaskService         | Application    | Fachliche Task-Operationen |
| TaskRepository      | Infrastructure | Speichern/Laden von Tasks  |
| NotificationService | Infrastructure | Benachrichtigungen         |
| ReminderService     | Application    | Erinnerungslogik           |

Alternativen: (Was haben wir verworfen und warum?)

- Alles in TaskMaster lassen, nur Methoden besser ordnen: behebt nicht das Problem mit der Testbarkeit und der Kopplung --> verworfen
- Nur Persistenz herauslösen: löst SRP nur zur Hälfte, Benachrichtigung und Reminder bleiben vermischte --> verworfen

Konsequenzen: (Was wird leichter? Was wird schwerer?)

- + Jede Klasse einzeln testbar, ein Änderungsgrund pro Klasse
- + self.cnt und self.email entfallen
- Mehr Klassen und mehr Dateien

## Refactoring Umsetzungsfälle

Umsetzungen sind im Projekt

- 1. Refactoring-vorschlag
  - Problem 1
  - TaskManager aufteilen in z. B.:
    - TaskService → fachliche Task-Operationen
    - TaskRepository → Speichern/Laden von Tasks
    - NotificationService → Benachrichtigungen
    - ReminderService → Erinnerungslogik
    - Ergänzend: UserService & UserRepository
- 2. Refactoring-Vorschlag
  - Problem 4, 5 & 8
  - Notifier-Port erstellen:
    - Strategy Pattern
- 3. Refactoring-Vorschlag
  - Problem 2
  - Composition Root + Konstruktor-Injektion
    - Erzeugung der Objekte nur noch in main
    - Alle anderen Klassen bekommen Abhängigkeiten als Konstruktorparameter und kennen nur die Ports

